List

In [2]:
```
l1 = [1,2,3,4,5]
```

In [4]:
```
print(l1)
```

```
[1, 2, 3, 4, 5]
```

In [6]:
```
print(type(l1))
```

```
<class 'list'>
```

In [8]:
```
l2 = [10, 2.5, True, 6+9j,"Hello"]
```

In [10]:
```
print(l2)
```

```
[10, 2.5, True, (6+9j), 'Hello']
```

In [12]:
```
l3 = [10,10,10,56.3,"Hi"]
```

In [14]:
```
print(l3)
```

```
[10, 10, 10, 56.3, 'Hi']
```

In [16]:
```
len(l2)
```

Out[16]:
```
5
```

In [18]:
```
for i in l2:
    print(i)
```

```
10
2.5
True
(6+9j)
Hello
```

In [20]:
```
l2[len(l2) - 1]
```

Out[20]:
```
'Hello'
```

In [22]:
```
l2[-1]
```

Out[22]:
```
'Hello'
```

In [24]:
```
for i in range(len(l2)):
    print(i, l2[i])
```

```
0 10
1 2.5
2 True
3 (6+9j)
4 Hello
```

In [26]:
```
l2[0] = 100
```

In [28]:
```
print(l2)
```

```
[100, 2.5, True, (6+9j), 'Hello']
```

In [30]:
```
l2
```

Out[30]:
```
[100, 2.5, True, (6+9j), 'Hello']
```

In [32]:
```
l3
```

Out[32]:
```
[10, 10, 10, 56.3, 'Hi']
```

In [34]:
```
l4 = l2 + l3
```

In [36]:
```
print(l4)
```

```
[100, 2.5, True, (6+9j), 'Hello', 10, 10, 10, 56.3, 'Hi']
```

In [38]:
```
l1
```

Out[38]:
```
[1, 2, 3, 4, 5]
```

In [40]:
```
print(l1 * 3)
```

```
[1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
```

In [42]:
```
200 in l2
```

Out[42]:
```
False
```

In [44]:
```
True in l2
```

Out[44]:
```
True
```

append(element), extend(iterable), insert(index, element), copy()

In [47]:
```
l1 = list()
```

In [49]:
```
l1
```

Out[49]:
```
[]
```

In [51]:
```
l2 = list("python")
```

In [53]:
```
l2
```

Out[53]:
```
['p', 'y', 't', 'h', 'o', 'n']
```

In [55]:
```
l3 = list(range(1,11))
```

In [57]:
```
l3
```

Out[57]:
```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

In [59]:
```
l1 = list(range(1,6))
```

In [61]:
```
l1
```

Out[61]:
```
[1, 2, 3, 4, 5]
```

In [63]:
```
l1.append(6)
```

In [65]:
```
l1
```

Out[65]:
```
[1, 2, 3, 4, 5, 6]
```

In [67]:
```
l1.append(7,8,9)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[67], line 1
----> 1 l1.append(7,8,9)

TypeError: list.append() takes exactly one argument (3 given)
```

In [69]:
```
l1.extend([7,8,9])
```

In [71]:
```
l1
```

Out[71]:
```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [79]:
```
l2 = [10,20,30]
l3 = [40,50]
```

In [81]:
```
l2 + l3
```

Out[81]:
```
[10, 20, 30, 40, 50]
```

In [83]:
```
l2
```

Out[83]:
```
[10, 20, 30]
```

In [85]:
```
l2.extend(l3)
```

In [87]:
```
l2
```

Out[87]:
```
[10, 20, 30, 40, 50]
```

In [89]:
```
l4 = ["hi", 6+5j]
print(id("hi"))
print(id(l4[0]))
```

```
140726548746720
140726548746720
```

In [91]:
```
l1
```

Out[91]:
```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [93]:
```
l1.insert(1,200)
```

In [95]:
```
l1
```

Out[95]: [1, 200, 2, 3, 4, 5, 6, 7, 8, 9]

In [97]:
```
len(l1)
```

Out[97]: 10

In [99]:
```
l1.insert(15,500)
```

In [101…
```
l1
```

Out[101… [1, 200, 2, 3, 4, 5, 6, 7, 8, 9, 500]

deep copy, shallow copy & reference assignment

In [104…
```
l1 = [10,20,30,40,[50,60,70],80]
```

In [106…
```
shallow_copy_l1 = l1.copy()
```

In [108…
```
shallow_copy_l1
```

Out[108… [10, 20, 30, 40, [50, 60, 70], 80]

In [110…
```
id(shallow_copy_l1)
```

Out[110… 2024385943168

In [112…
```
id(l1)
```

Out[112… 2024355560128

In [114…
```
l1.append(90)
```

In [116…
```
l1
```

Out[116… [10, 20, 30, 40, [50, 60, 70], 80, 90]

In [118…
```
shallow_copy_l1
```

Out[118… [10, 20, 30, 40, [50, 60, 70], 80]

In [120…
```
l1[4]
```

Out[120… [50, 60, 70]

In [122…
```
l1[4].append("hi")
```

In [124…
```
l1
```

Out[124… [10, 20, 30, 40, [50, 60, 70, 'hi'], 80, 90]

In [126…
```
shallow_copy_l1
```

Out[126… [10, 20, 30, 40, [50, 60, 70, 'hi'], 80]

In [128…
```
# shallow_copy_l1 = l1.copy()
# shallow_copy_l1 = l1[:]
# shallow_copy_l1 = list(l1)
```

In [130…
```
l1 = [10,20,30,40,[50,60,70],80]
```

In [132…
```
import copy
deep_copy_l1 = copy.deepcopy(l1)
```

In [134…
```
l1
```

Out[134…
```
[10, 20, 30, 40, [50, 60, 70], 80]
```

In [136…
```
deep_copy_l1
```

Out[136…
```
[10, 20, 30, 40, [50, 60, 70], 80]
```

In [138…
```
l1.append(90)
```

In [140…
```
l1
```

Out[140…
```
[10, 20, 30, 40, [50, 60, 70], 80, 90]
```

In [142…
```
deep_copy_l1
```

Out[142…
```
[10, 20, 30, 40, [50, 60, 70], 80]
```

In [144…
```
l1[4].append("Hi")
```

In [146…
```
l1
```

Out[146…
```
[10, 20, 30, 40, [50, 60, 70, 'Hi'], 80, 90]
```

In [148…
```
deep_copy_l1
```

Out[148…
```
[10, 20, 30, 40, [50, 60, 70], 80]
```

In [150…
```
l1 = [10,20,30,40,[50,60,70],80]
```

In [154…
```
ref_l1 = l1
```

In [156…
```
print(id(l1))
print(id(ref_l1))
```

```
2024385800640
2024385800640
```

In [158…
```
l1.append(90)
l1[4].append("Hi")
```

In [160…
```
l1
```

Out[160…
```
[10, 20, 30, 40, [50, 60, 70, 'Hi'], 80, 90]
```

In [162…
```
ref_l1
```

Out[162… [10, 20, 30, 40, [50, 60, 70, 'Hi'], 80, 90]

Identity Operator : is & is not

In [165…
```
l4 = [1,2,3,4]
```

In [167…
```
l5 = [1,2,3,4]
```

In [169…
```
print(l4 == l5)
```

```
True
```

In [171…
```
print(l4 is l5)
```

```
False
```

In [179…
```
print(l4 is not l5)
```

```
True
```

In [173…
```
l6 = l4
```

In [175…
```
print(l6 == l4)
```

```
True
```

In [177…
```
print(l6 is l4)
```

```
True
```

In [185…
```
x = 10 # -5 to 255
y = 10
print(x is y)
```

```
True
```

In [187…
```
x = int(input())
y = int(input())
print(x is y)
```

```
True
```

In [191…
```
x = 2.5
y = 2.5
print(x is y)
```

```
False
```

In [193…
```
x = 6+5j
y = 6+5j
print(x is y)
```

```
False
```

In [195…
```
x = True
y = True
print(x is y)
```

```
True
```

In [197…
```
x = "hi"
y = "hi"
print(x is y)
```

```
True
```

In [199…
```
s1 = "welcome"
s2 = "welcome"
print(s1 is s2)
```

```
True
```

In [205…
```
s1 = input()
s3 = input()
print(s1 is s3)
```

```
False
```

pop([index]), remove(element), clear() , del

In [208…
```
l1
```

Out[208…
```
[10, 20, 30, 40, [50, 60, 70, 'Hi'], 80, 90]
```

In [210…
```
l1.pop()
```

Out[210…
```
90
```

In [212…
```
l1.pop()
```

Out[212…
```
80
```

In [214…
```
l1.pop(1)
```

Out[214…
```
20
```

In [216…
```
l1
```

Out[216…
```
[10, 30, 40, [50, 60, 70, 'Hi']]
```

In [218…
```
l1.remove(10)
```

In [220…
```
l1
```

Out[220…
```
[30, 40, [50, 60, 70, 'Hi']]
```

In [222…
```
l1.remove(100)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[222], line 1
----> 1 l1.remove(100)

ValueError: list.remove(x): x not in list
```

In [224…
```
l1.clear()
```

In [226…
```
print(l1)
```

```
[]
```

In [228…
```
del l1
```

In [230…
```
print(l1)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[230], line 1
----> 1 print(l1)

NameError: name 'l1' is not defined
```

count(element), revese(), sort(*, key = None, reverse = False)

In [233…
```
l1 = list(range(10,51,10))
```

In [235…
```
l1
```

Out[235…
```
[10, 20, 30, 40, 50]
```

In [237…
```
l1.count(100)
```

Out[237…
```
0
```

In [239…
```
l1
```

Out[239…
```
[10, 20, 30, 40, 50]
```

In [241…
```
l1[::-1]
```

Out[241…
```
[50, 40, 30, 20, 10]
```

In [243…
```
l1
```

Out[243…
```
[10, 20, 30, 40, 50]
```

In [245…
```
l1.reverse()
```

In [247…
```
l1
```

Out[247…
```
[50, 40, 30, 20, 10]
```

In [251…
```
l1.sort()
```

In [253…
```
l1
```

Out[253…
```
[10, 20, 30, 40, 50]
```

In [255…
```
l1.sort(reverse=True)
```

In [257…
```
l1
```

Out[257… [50, 40, 30, 20, 10]

In [259…
```
l = ["cat", "Catch", "Bat", "apple", "Dog"]
```

In [261…
```
l.sort()
```

In [263…
```
l
```

Out[263… ['Bat', 'Catch', 'Dog', 'apple', 'cat']

In [265…
```
l.sort(key=str.lower)
```

In [267…
```
l
```

Out[267… ['apple', 'Bat', 'cat', 'Catch', 'Dog']

In [269…
```
l.sort(key = str.lower, reverse=True)
```

In [271…
```
l
```

Out[271… ['Dog', 'Catch', 'cat', 'Bat', 'apple']

In [273…
```
l.sort(key = len)
```

In [275…
```
l
```

Out[275… ['Dog', 'cat', 'Bat', 'Catch', 'apple']

In [277…
```
l.sort(key = len, reverse= True)
```

In [279…
```
l
```

Out[279… ['Catch', 'apple', 'Dog', 'cat', 'Bat']

List Comprehension

In [282…
```
l = [1,2,3,4,5]
```

In [284…
```
res = []
for i in l:
    res.append(i**2)
print(res)
```

```
[1, 4, 9, 16, 25]
```

In [286…
```
# res = [exp for i in iterable]
```

In [288…
```
res1 = [i**2 for i in l]
```

In [290…
```
res1
```

Out[290… [1, 4, 9, 16, 25]

In [292…

```
l = [10,15,20,25,30]
res = [i for i in l if i %2 == 0]
print(res)
```

```
[10, 20, 30]
```

In [296…

```
l = [1,2,3,4,5]
res = []
for i in l:
    if i%2 == 0:
        res.append("Even")
    else:
        res.append("Odd")
print(res)
```

```
['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

In [300…

```
l = [1,2,3,4,5]
ans = ["Even" if i%2 == 0 else "Odd" for i in l]
print(ans)
```

```
['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

Problem Definition: Replace odd numbers with 0

Sample Input:

```
[1, 2, 3, 4]
```

Sample Output:

```
[0, 2, 0, 4]
```

In [306…

```
l = [1,2,3,4]
ans = [i if i%2 == 0 else 0 for i in l]
ans
```

Out[306…

```
[0, 2, 0, 4]
```

Problem Definition: Extract digits from string

Sample Input:

```
"a1b2c3"
```

Sample Output:

```
['1', '2', '3']
```

In [309…

```
print([i for i in "a1b2c3" if i.isdigit()])
```

```
['1', '2', '3']
```

Problem Definition: Words starting with vowel

Sample Input:

```
["apple", "banana", "orange", "grape"]
```

Sample Output:

```
['apple', 'orange']
```

In [312…

```
vowel = "aeiouAEIOU"
l = ["apple", "banana", "orange", "grape"]
ans = [i for i in l if i[0] in vowel]
ans
```

Out[312…

```
['apple', 'orange']
```

Problem Definition: Flatten 2D list

Sample Input:

```
[[1, 2], [3, 4], [5, 6]]
```

Sample Output:

```
[1, 2, 3, 4, 5, 6]
```

In [315…

```
l = [[1, 2], [3, 4], [5, 6]]
ans = [ele for i in l for ele in i]
ans
```

Out[315…

```
[1, 2, 3, 4, 5, 6]
```

In [317…

```
# for i in l:
#     for ele in i:
#         print(ele)
```

Problem Definition: Classify numbers

Sample Input:

```
[-2, -1, 0, 1, 2]
```

Sample Output:

```
['Negative', 'Negative', 'Zero', 'Positive', 'Positive']
```

In [320…

```
l = [-2, -1, 0, 1, 2]
ans = ["Positive" if i > 0 else "Zero" if i == 0 else "Negative" for i in l]
ans
```

Out[320…

```
['Negative', 'Negative', 'Zero', 'Positive', 'Positive']
```

Problem Definition: Words starting with digit

Sample Input:

```
["1abc", "abc", "9xyz"]
```

Sample Output:

```
['1abc', '9xyz']
```

In [323…

```
l = ["1abc", "abc", "9xyz"]
ans = [i for i in l if i[0].isdigit()]
ans
```

Out[323…

```
['1abc', '9xyz']
```

Problem Definition: CSV string to integers

Sample Input:

```
"78, 85, 90"
```

Sample Output:

```
[78, 85, 90]
```

In [326…

```
s = "78, 85, 90"
l = [int(i) for i in s.split(", ")]
l
```

Out[326…

```
[78, 85, 90]
```

Problem Definition: Remove special characters

Sample Input:

```
"ab@#12$c"
```

Sample Output:

```
['a', 'b', '1', '2', 'c']
```

In [331…

```
l = [i for i in "ab@#12$c" if i.isalnum()]
l
```

Out[331…

```
['a', 'b', '1', '2', 'c']
```

In [ ]:

```
```